

Monte Carlo π and Markov Chain Monte Carlo

Assignment #1

Objective: Compute the estimate of simple integrals using Monte Carlo methods for uncorrelated and correlated samples. Become familiar with the use of an hpc environment for running simulations

Activity 1: Monte Carlo estimate of π

Compute an estimate of π using the area method and the direct sampling of the integral using a MC estimator.

Comment the convergence of the results for each of the two methods based on the number of Monte Carlo samples used in the simulation. Discuss how π estimate change depending on the chosen method (hint: make a table with a column for the number of samples and a column with the estimate obtained for each method).

Tasks

The code `area_method.py` estimate π counting the ratio between the number of data points generated in the quarter circle (N_c) with respect to the total one (N_c/N). Provide the number of points from command line with:

```
python3 area_method.py -N 50000.
```

Repeat the process for different N to study the convergence of the estimate depending on the number of samples.

The code `mc_estimator.py` samples directly the function $f(x) = \sqrt{1-x^2}$. The area of the quarter circle $A_c = \int_0^1 f(x)dx$ can be rewritten as the expectation value of $f(x)/p(x)$ with respect to the probability distribution $p(x)$, namely $A_c = \int_0^1 \frac{f(x)}{p(x)} p(x) dx$. This simple trick allow to introduce an MC estimator for the discrete case of N samples $\langle f(x)/p(x) \rangle_p \sim 1/N \sum_{i=1}^N f(x_i)/p(x_i)$. The number of points for the simulation are provided from command line: `python3 area_method.py -N 50000`.

Repeat the process for different N and study the convergence of the MC estimator on the number of samples. How does the estimate changes between the two different methods and by increasing the number of samples?

Plot the converged results with `plot.py`. You can plot either during the convergence for a single method (e.g. mc estimator) to inspect in real-time or after the convergence is finished using for both codes `python3 plot.py -mode mc` or `python3 plot.py -mode both`.

Moving to repeated measurements for the estimate π there is the need to use different sequences of random numbers, so we do not fix the seed anymore but we rely on independent sequences of random numbers. Repeat M measurement with the optimal N value found previously for each of the two methods using the code `histogram.py`. Plot the histogram of the distribution for π -estimate obtained doing M experiment for each of the two methods e.g. `python3 histogram.py -N 100000 -M 500`. How do mean and variance of π -estimate changes comparing the area method and the mc estimator, why?

Hints / Instructions

Both the code `area_method.py` and `mc_estimator.py` append an array [N pi-estimate] on a .txt file which is then read from `plot.py` to make the convergence plot. Consider to clean these files when you want to perform a new convergence.

The scripts rely on which is a command line parser used in python. Options like the number of samples can be passed with the `-option` as shown before.

For each of the codes provided it is possible to show the options it accepts by typing `python3 code_name.py -help`

The codes `area_method.py` and `mc_estimator.py` use simple implementations of functions with `for loops` and more efficient sampling techniques `vectorized functions`. To show that the implementations are consistent and reproduce the same results during the convergence test we fix the `seed` of the random number generator.

Activity 2: Compute a bimodal Gaussian distribution using Markov Chain Monte Carlo

Markov Chain Monte Carlo (MCMC) is used to sample probability distributions given valid rules (updates) that allow a system to move ergodically in the configurational space. The updates can be accepted or rejected according to the acceptance probabilities computed with the detailed balance equation. Monitor the ratio of accepted and rejected updates by varying the number of samples, study the convergence of the sampled distribution (histogram) depending on the number of samples. Repeat the analysis varying the parameters of the distribution (x_0 , σ , $[x_{min}, x_{max}]$).

Tasks

The code `bimodal.py` performs a MCMC simulation updating the current state x according to a bimodal gaussian probability distribution. A new state is sampled from a uniform distribution between $[x_{min}, x_{max}]$ where $p(x) = 1/(x_{max} - x_{min})$ and the acceptance probability is calculated according to the detailed balance equation from Metropolis-Hastings $A_{acc} = \min(1, \frac{f(x_{prop})}{f(x_{curr})})$. The code output the ratio of accepted and rejected updates to monitor during the simulations.

Inspect the command line argument that can be passed to initialize the parameters of the distribution (gaussian centers, std), its domain (x_{min}, x_{max}) and select the number of samples both for the thermalization of the Markov Chain and for the run. `python3 bimodal.py -help`

How does the ratio of the accepted and rejected updates converge to a stable value while increasing the number of samples? Compute a convergence test changing `-N` for the bimodal gaussian while keeping the default values provided by the code for the other command line parameters. (hint: report the values in a table as a function of the number of samples). Do you think a converged acceptance ratio is enough, in general, to determine a correct and converged sampling of a probability distribution?

Report as a metric for the sampling convergence the MSE of the histogram values with respect to the analytic function. Is the MSE a good metric in this case or is it can be affected by a certain simulation parameter?

Repeat the simulation and the analysis of the previous steps initializing manually the parameters of the distribution and the ones related to the sampling for a couple of other examples (more if you want and you like it). e.g. : `python3 bimodal.py -x0 3 -sigma 1 -xmax 10 -xmin -10 -N 50000 -N_therm 5000 -n_bins 100`.

Once you are at convergence with samples, do you notice some dependencies of the ratio of accepted updates depending on the parameters used to initialize the bimodal gaussian and its shape?

Hints / Instructions

MCMC deals with correlated samples, consequently gaussian statistics fails. Therefore, a parameter affecting the goodness of results is `n_bins`. The MSE is, therefore, replaced by a more robust branch of statistical analysis called **blocking analysis** that will be used in the Diagrammatic Monte Carlo codes of the next lab days.

The code has more input parameters, check how to initialize them correctly e.g. a combination of parameters such as `-x0 2 -sigma 1 -x_max 4 -x_min -4` will sample a truncated bimodal gaussian because the sampling range is cutting the tails beyond 2σ . It's still fine, just be aware of what you want to study and observe.

Enjoy the lab!

Cesare Franchini, Marco Barducci, Davide Patricelli